

k6

Open-source load/performance testing tool with JavaScript-based scripts.

QA use

Supports load models, thresholds and scenarios to evaluate service and application behavior and indicators.

Official site and documentation

Use the official k6 source as the primary reference: <https://grafana.com/oss/k6/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Metrics

Analyze latency, percentiles, error rate, throughput and infrastructure behavior together with test results.

Strategy

Performance testing needs an objective, load model, representative environment and clear criteria — not merely a high request volume.

When to use it

Use this knowledge when product risk, architecture or the delivery stage requires this type of validation. Confirm the goal, environment, required data and expected evidence before starting.

How QA applies it

Start from the behavior or risk to validate, prepare data and environment, execute the technique or tool reproducibly and capture evidence that helps the team understand the result.

Good practices

Prioritize high-impact scenarios, keep data and environments controlled, record important decisions and review the approach periodically to avoid redundant or outdated coverage.

What it is and where it fits

k6 is a tool or technology that can support Quality Engineering work. Before adopting it, understand the problem it solves, where it fits in the workflow and what evidence or productivity gain it should provide.

First QA-oriented use

Pick one small, verifiable task in k6, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where k6 fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether k6 still delivers value as the product evolves.

Common issues and troubleshooting

When k6 fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.